Cynbit: AI Opportunity Report and Technical Approach

AI/ML Engineer Intern task. Prepared from Cynbit's public website (home page and product suite page).

Part 1: AI Opportunity Report

1. What I looked at

I went through the Cynbit home page and the product suite page, and read them the way a new hire would on day one: what does the company sell, who pays, and where does the work still depend on people doing repetitive things.

Cynbit is a Jaipur company that does three things at once. It builds custom software and AI automation for clients (web, mobile, cloud, technical SEO). It runs its own SaaS products: Cynbit Studio for career preparation, TenantEase for property and rent management, and Revio for review and reputation management. And it runs BitElevate, a training program that the site says has certified 1000+ people. Their own pitch is "AI-first thinking", with a six-step delivery process that starts with discovery and an audit.

2. Where AI already shows up, and where it doesn't

This matters because a good suggestion shouldn't repeat what they already have.

- **Cynbit Studio** already has AI mock interviews, AI code evaluation, an AI mentor and a resume engine. It is the most AI-heavy product.
- **Revio** is AI by definition: sentiment analysis, auto-generated review replies, crisis detection.
- **TenantEase** is the odd one out. Its page describes unit tracking, tenant profiles, a dashboard and "system-driven" rent reminders. Nothing there mentions prediction or learning. It records what happened and sends the same style of reminder to everyone.
- **Services and delivery**: the "Discovery & audit" step is manual, and the site doesn't mention any tooling behind it.

3. Opportunities I considered

Opportunity	Where	Value	Effort	Comment
Rent-risk prediction and smart reminders	TenantEase	High	Low to medium	Clear business metric (collection rate), data already exists inside the product, no AI there today
Lead scoring and proposal drafting from discovery calls	Services	Medium	Medium	Useful, but needs call data I can't see, hard to prove without it
Churn and dropout prediction	Studio / BitElevate	Medium	Medium	Good idea, but Studio already has a lot of AI and I'd be guessing at usage data
Voice-of-customer clustering across Revio reviews	Revio	Medium	Medium	Natural extension, but Revio's value is already the AI layer, so the gap is smaller
Cross-product recommendation (Studio learners to BitElevate)	Studio / BitElevate	Low to medium	Low	Easy, but modest impact

4. The one I'm recommending: rent-risk prediction for TenantEase

The problem. For a small landlord, late rent is the daily headache. TenantEase already tracks dues and sends reminders, so it has the history needed to do something smarter, but today every tenant gets the same treatment. A tenant who has paid on the 1st for three years and a tenant who has been late five months running get the same message at the same time.

The idea. Look at each tenant's payment history and a few simple facts (how they pay, whether autopay is on, how long they've lived there, the month) and estimate the chance they'll pay late next cycle. Then use that score to decide who gets an early nudge, who gets a softer note, and who the landlord should actually call. Every score comes with a plain-language reason, such as "paid late in 5 of the last 6 months", so a landlord can trust it or overrule it.

Why this one.

1. It fits the product. It is a feature a property manager would see on the dashboard and understand in ten seconds.
2. The target is easy to measure: collection rate and days-to-pay.
3. It is a real ML problem (tabular, imbalanced, needs explanation) without needing anything exotic or expensive to run.
4. It is a gap. Studio and Revio already lean on AI. TenantEase is where the product story is thinnest.
5. It can be sold. "Higher on-time collection" is a line a landlord pays for, and it can sit in a premium tier.

Who benefits. Landlords and property managers (less chasing, steadier cash flow), tenants (fewer awkward calls, reminders that fit their situation), and Cynbit (a reason to upsell TenantEase and a data asset that gets better with more properties).

5. How we'd know it worked

- On-time payment rate in the first 5 days after due date, compared with a control group that keeps the current reminders.
- Average days to payment.
- Share of landlord follow-up calls that were actually needed (precision of the high-risk list).
- Reminder opt-out or complaint rate, as a guardrail so we don't annoy people.

6. Risks and honest limits

- **I don't have Cynbit's real data.** The proof of concept uses synthetic data built to behave plausibly. It proves the pipeline, not the accuracy. Real numbers could be better or worse.

- **Fairness.** Features like occupation can end up penalising groups of people. I'd keep them out of any decision that affects a tenant directly, and only use risk to change *when and how* a reminder is sent, never to deny anything.
- **Privacy.** Tenant ID details and payment history are personal data. The model needs only payment behaviour, not ID documents.
- **Cold start.** A new landlord with no history won't get good scores. Fall back to simple rules until there are a few months of data.

7. Next steps

1. Ask the TenantEase team for an anonymised export of payment history (even a few hundred tenants).
2. Re-train and check whether the lift over the simple rule survives on real data.
3. Pilot with a few friendly landlords, with a control group.
4. Decide, based on that result, whether it ships as a core feature or a paid add-on.

Part 2: Technical Approach Report

1. The task in ML terms

Binary classification. For each tenant, at the start of a rent cycle, predict whether payment will arrive more than a few days after the due date. The output is a probability, not just a yes or no, because the business wants a ranking (who to chase first) and bands (low, medium, high).

2. Data

TenantEase should already hold most of this: rent amount, due day, payment timestamps, payment method, tenure, reminders sent, and whether ID was verified.

Features I used in the proof of concept:

- Behaviour: late payments in the last 6 months, average days late, reminders needed last month, autopay on or off.
- Contract: rent, deposit in months, tenure in months, ID verified.
- Context: payment method, occupation, festive month.

Target: paid late in the next cycle (1 or 0).

Synthetic data. Because the real data isn't available, I generated 4,000 tenants. A hidden "reliability" value drives both past lateness and next-month lateness, and a few direct effects are added (cash payers and no-autopay tenants are riskier, students and self-employed slightly more so, festive months push risk up). About 20% end up late, which is a plausible imbalance. The model never sees the hidden value.

3. Model choice

- **Baseline:** rank tenants by how many times they were late in the last 6 months. This is what a sensible person would do without ML, so the model has to beat it.
- **Model:** gradient boosting (scikit-learn `HistGradientBoostingClassifier`, 150 trees, depth 4). It handles mixed tabular features and non-linear effects without much tuning, trains in seconds, and runs on a CPU.
- **Why not deep learning or an LLM here:** the data is a small table. A neural network adds cost and opacity and no accuracy. LLMs are for the message-writing step only.

I split 80/20 with stratification and only score the held-out 20% in the demo, so the app never shows predictions on data the model trained on.

4. Results from the prototype

Metric	Past-lateness rule	Gradient boosting
ROC-AUC	0.634	0.712
PR-AUC	0.287	0.383
Precision at top 20%	0.344	0.394
Recall at top 20%	0.344	0.394

The base late rate is 20%. Chasing the top 20% of tenants by model score finds late payers about 39% of the time, against 34% for the simple rule and 20% for picking at random. The lift is real but modest. I would rather report that than inflate it: on synthetic data the number mostly reflects how I built the data, so the useful result is that the pipeline works end to end and beats the baseline, and the real test is on Cynbit's own data.

5. Explainability

Landlords won't act on a black-box score. For every tenant the app takes each feature group, replaces it with the typical value, and measures how much the predicted risk falls. The biggest drops become the plain-English reasons shown next to the score. This is simple, runs in one batch prediction, and doesn't need extra libraries. If the team wants something more formal, SHAP is the natural upgrade.

6. Turning a score into an action

Band	Risk	What happens
Low	under 30%	Short reminder the day before
Medium	30% to 50%	Friendly reminder about 3 days early
High	50% and above	Reminder about 5 days early, invites the tenant to say if there's a problem, flags the landlord to follow up

Messages come from templates in English or Hinglish, since that is how many landlords actually write to tenants. An optional LLM step (Claude, via API) can rewrite a template so it sounds natural. It is deliberately a small, cheap call on a short prompt: the facts (amount, date) are passed in and told not to change, and the LLM never decides who gets contacted. That decision stays with the model and the landlord.

7. Architecture

1. **Data layer:** TenantEase database, with a nightly job that builds one feature row per tenant per upcoming cycle.
2. **Scoring service:** a small API (FastAPI or a scheduled job) that loads the saved model and writes a risk score and reasons back to the tenant record.
3. **Reminder engine:** reads the score, picks the band and timing, renders the message, and sends it through the channels TenantEase already uses.
4. **Dashboard:** shows risk bands, expected rent at risk, and the reasons.
5. **Monitoring:** log the score, the action taken and the actual outcome, so we can measure lift and spot drift.

8. Prompt design for the optional LLM step

The prompt is short and tight: rewrite the reminder warmly, under 60 words, keep the amount and day unchanged, reply with the message only. Nothing about the tenant's risk score goes into the prompt, so the LLM can't produce a message that sounds accusatory. The smallest available Claude model is enough, and the call only happens when a landlord asks for a rewrite or when messages are batch-generated once a month, so cost stays small.

9. Execution plan

Phase	What	Rough time
1	Get a data export, define "late", clean it, rerun the notebook on real data	Week 1
2	Tune the model, check calibration, compare to the baseline on real data	Week 2
3	Build the scoring job and reminder logic, add the dashboard widget	Weeks 3 and 4
4	Pilot with a few landlords and a control group	Weeks 5 to 8
5	Review results, decide on rollout and pricing	Week 9

10. How to run the prototype

```
pip install pandas numpy scikit-learn streamlit
streamlit run tenantease_risk_poc.py
```

Running `python tenantease_risk_poc.py` without Streamlit prints the metrics and the five riskiest tenants in the terminal. To try the LLM rewrite, set `ANTHROPIC_API_KEY` first (and `pip install anthropic`).

11. What I'd do differently with more time

- Calibrate the probabilities (so "60%" really means about 60%).
- Use time-based splits instead of random ones, since real data has a time order.

- Add a survival-style view (how many days late, not just late or not).
- Check the model across groups of tenants for uneven error rates before any rollout.